

DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY

Object Oriented Programming with Java

Question Bank - Solved Answers

Note: Answers are written in AKTU exam format as per the syllabus and standard question bank.

SECTION - A (Short Answer Questions)

(Each question carries 2 Marks)

Q1. Define Object Oriented Programming.

Object Oriented Programming (OOP) is a programming paradigm that organizes software design around data (objects) rather than functions and logic. An object is a self-contained unit that consists of both data (attributes/fields) and behavior (methods/functions). OOP is based on four main principles: Encapsulation, Inheritance, Polymorphism, and Abstraction. Languages like Java, C++, and Python follow the OOP paradigm.

Q2. Briefly define Encapsulation.

Encapsulation is the process of wrapping data (variables) and the code that operates on it (methods) together as a single unit, i.e., a class. It hides the internal details of an object from the outside world and only exposes necessary functionality through public methods (getters/setters). This concept is also known as data hiding. Example: Making class variables private and providing public getter/setter methods to access them.

Q3. Define Classes under Basic Structural Modeling.

In Basic Structural Modeling (UML), a Class is a blueprint or template that describes the attributes (data) and operations (behaviors) shared by a group of objects. A class is represented as a rectangle divided into three compartments: the top compartment contains the class name, the middle contains attributes, and the bottom contains operations/methods. Classes are the fundamental building blocks of object-oriented systems.

Q4. Discuss about Object Diagrams.

An Object Diagram is a UML diagram that represents a snapshot of the objects in a system at a specific moment in time. It shows the instances of classes and their relationships at runtime. Object diagrams are used to verify class diagrams by showing actual object values. Notation: An object is shown as a rectangle with the format `objectName:ClassName`. Attribute values are shown in the middle compartment. Object diagrams help in understanding the system behavior during execution.

Q5. Briefly discuss about Object Oriented Analysis (OOA).

Object Oriented Analysis (OOA) is the process of examining a problem domain to understand the requirements and model the real-world entities as objects. It focuses on identifying objects, their attributes, relationships, and interactions. The main activities in OOA include: identifying classes and objects, defining attributes and operations, identifying relationships (associations, inheritance, aggregation), and building use-case models. OOA uses techniques like OMT, OOAD, and UML for modeling.

Q6. What do you mean by Physical Packaging?

Physical Packaging in OOP refers to the organization of classes and components into physical units such as files, libraries, or packages for deployment purposes. In Java, physical packaging is done using the 'package' keyword. It groups related classes and interfaces together. Physical packaging helps in: modular development, avoiding name conflicts, providing access control, and making the code reusable. Example: java.util, java.io are standard Java packages.

Q7. What is the use of Scope Resolution Operator?

The Scope Resolution Operator (::) is used in C++ to define the scope (namespace or class) to which a function or variable belongs. It is used to: (i) Define a member function outside its class declaration, (ii) Access static members of a class, (iii) Resolve ambiguity in multiple inheritance, (iv) Access global variables when a local variable with the same name exists. Note: Java does not use the scope resolution operator; instead it uses the dot operator (.) for similar purposes.

Q8. Define Identifiers.

Identifiers are names given to various programming elements like variables, methods, classes, packages, and interfaces in Java. Rules for identifiers in Java: (i) Can consist of letters, digits, underscore (_), and dollar sign (\$). (ii) Must start with a letter, underscore, or dollar sign - not a digit. (iii) Cannot be a reserved keyword. (iv) Are case-sensitive (e.g., 'count' and 'Count' are different). (v) No limit on length. Example: int studentAge; - here 'studentAge' is an identifier.

Q9. What do you understand by Private and Public members?

Private members: Variables or methods declared with the 'private' access specifier are accessible only within the same class. They cannot be accessed by any outside class. This enforces encapsulation. Example: private int salary;

Public members: Variables or methods declared with the 'public' access specifier are accessible from anywhere - within the class, outside the class, in different packages, and in subclasses. Example: public void display() {}

Private members ensure data security while public members provide the interface for interaction with the object.

Q10. Discuss Constructors and their types.

A Constructor is a special method in Java that is called automatically when an object of a class is created. It has the same name as the class and no return type.

Types of Constructors:

- Default Constructor: Takes no parameters. Java provides it automatically if no constructor is defined. Example: Student() {}
- Parameterized Constructor: Accepts arguments to initialize object with specific values. Example: Student(int id, String name) {}
- Copy Constructor: Creates a new object as a copy of an existing object. Java does not provide this automatically; programmer must define it.

Q11. Differentiate between print() and println() methods in Java.

print() method: Prints the output on the same line without moving the cursor to the next line. Subsequent output continues on the same line. Example: System.out.print("Hello"); System.out.print("World"); Output: HelloWorld

println() method: Prints the output and then moves the cursor to the next line (adds a newline character at the end). Example: System.out.println("Hello"); System.out.println("World"); Output: Hello (newline) World

Q12. What are Symbolic Constants? Explain with examples.

Symbolic Constants are named constants defined using the 'final' keyword in Java (or #define in C++). They give a meaningful name to a fixed value that cannot be changed during program execution.

Advantages: Improves code readability, makes modification easier, and reduces errors.

Example in Java:

```
final double PI = 3.14159;  
final int MAX_SIZE = 100;
```

Once assigned, the value of a symbolic constant cannot be modified. Attempting to reassign causes a compile-time error.

Q13. What is the significance of the CLASSPATH environment variable?

CLASSPATH is an environment variable that tells the Java Virtual Machine (JVM) and Java compiler where to find the compiled .class files (bytecode) of user-defined classes and packages. When you create a package, the CLASSPATH must include the directory containing the package's root directory so that Java can locate and load the classes. Without proper CLASSPATH settings, Java will throw a ClassNotFoundException or NoClassDefFoundError. In modern IDEs and build tools (Maven, Gradle), CLASSPATH is managed automatically.

Q14. What is the difference between Error and Exception?

Error: An Error indicates a serious problem that a reasonable application should not try to catch. Errors are typically caused by the environment in which the application is running (e.g., OutOfMemoryError, StackOverflowError). Errors are instances of java.lang.Error class.

Exception: An Exception is a problem that can be caught and handled by the program. Exceptions can be anticipated and managed using try-catch blocks. Exceptions are instances of java.lang.Exception class. Example: IOException, NullPointerException, ArrayIndexOutOfBoundsException.

Q15. What is Synchronization and why is it important?

Synchronization in Java is a mechanism that ensures that only one thread can access a shared resource (critical section) at a time. It prevents data inconsistency caused by concurrent access by multiple threads. Synchronization is important because: (i) It ensures thread safety, (ii) Prevents race conditions, (iii) Maintains data integrity in multi-threaded applications. In Java, synchronization is achieved using the 'synchronized' keyword on methods or blocks.

```
synchronized void updateBalance(int amount) { balance += amount; }
```

Q16. What is the significance of Legacy class? Give example.

Legacy classes are the original collection classes that were part of Java before the Collections Framework was introduced in Java 2 (JDK 1.2). They were redesigned to support generics and are now part of the Collections Framework. Legacy classes include: Vector, Hashtable, Stack, Properties, Dictionary (abstract). Example: Vector is similar to ArrayList but is synchronized. Hashtable is similar to HashMap but is synchronized. They are retained for backward compatibility.

Q17. What are the differences between JToggleButton and RadioButton?

JToggleButton: A two-state button that can be in ON or OFF state. When clicked once it is selected; when clicked again it is deselected. Multiple JToggleButtons can be selected simultaneously unless grouped. It is a standalone component.

JRadioButton: A JRadioButton extends JToggleButton but is typically used in a ButtonGroup where only one radio button can be selected at a time (mutual exclusion). Radio buttons are used when the user must choose exactly one option from a set of options.

Q18. What is an Adapter class? Explain with an example.

An Adapter class in Java (AWT/Swing) is an abstract class that provides default (empty) implementations of all methods defined in a listener interface. Instead of implementing every method of an interface, a programmer can extend the adapter class and override only the required methods.

Example: MouseAdapter implements all methods of MouseListener. If you only need mouseClicked():

```
class MyMouse extends MouseAdapter {
    public void mouseClicked(MouseEvent e) {
        System.out.println("Mouse Clicked");
    }
}
```

Q19. What is an Abstract class? Give example.

An abstract class is a class declared with the 'abstract' keyword that cannot be instantiated on its own. It may contain abstract methods (methods without body) that must be implemented by its subclasses, as well as concrete methods with implementation.

Example:

```
abstract class Shape {
    abstract double area(); // abstract method
    void display() { System.out.println("I am a shape"); }
}
class Circle extends Shape {
    double r;
    Circle(double r) { this.r = r; }
    double area() { return 3.14 * r * r; }
}
```

Q20. Write the complete life cycle of a thread.

A thread in Java goes through the following states during its lifecycle:

1. New: Thread object is created but start() has not been called.
2. Runnable: Thread is ready to run after start() is called. The thread scheduler picks it.
3. Running: Thread is currently executing.
4. Blocked/Waiting: Thread is waiting for a resource (I/O, lock, or another thread to notify).
5. Timed Waiting: Thread is waiting for a specified time period (sleep(), join(timeout)).
6. Terminated (Dead): Thread has completed execution or was stopped.

Q21. What is the benefit of Generics in Collections Framework?

Generics allow type safety in Java Collections by specifying the type of elements a collection can hold at compile time, rather than at runtime.

Benefits:

- Type Safety: Prevents ClassCastException by detecting type mismatches at compile time.
- Elimination of Casting: No explicit type casting needed while retrieving elements.
- Reusability: Generic classes/methods work with any data type.

- Code Clarity: Makes code more readable and self-documenting.

Example: `ArrayList<String> list = new ArrayList<String>();` ensures only String objects can be stored.

Q22. Differentiate between Enumeration and Iterator interface.

Enumeration: An older interface (legacy) with two methods - `hasMoreElements()` and `nextElement()`. It does not allow removal of elements during traversal. It is used with legacy classes like Vector and Hashtable.

Iterator: A part of the Collections Framework with three methods - `hasNext()`, `next()`, and `remove()`. It allows removal of elements during traversal using `remove()`. It works with all Collection classes. Iterator is preferred over Enumeration for modern Java code.

SECTION - B (Medium Answer Questions)

(Each question carries 7 Marks)

Q1. Differentiate between OOP and Procedural Programming.

Procedural Programming: A programming paradigm based on the concept of procedure calls. Programs are a sequence of instructions grouped into functions/procedures. Data and functions are separate. Emphasis is on 'how to do'. Example languages: C, Pascal, BASIC.

Object Oriented Programming: A programming paradigm based on objects that combine data and behavior. Emphasis is on 'what to do'. Uses classes and objects. Provides features like inheritance, polymorphism, encapsulation, and abstraction. Example languages: Java, C++, Python.

Key Differences:

- OOP uses classes; Procedural uses functions/procedures.
- OOP supports inheritance and polymorphism; Procedural does not.
- OOP provides data hiding via encapsulation; Procedural data is globally accessible.
- OOP is better for large, complex applications; Procedural suits simple, sequential tasks.
- OOP promotes code reuse via inheritance; Procedural promotes reuse via function calls.

Q2. Explain in detail the concepts and types of Inheritance.

Inheritance is an OOP mechanism where a new class (subclass/child) acquires the properties and behaviors of an existing class (superclass/parent). It promotes code reuse and establishes an IS-A relationship. In Java, inheritance is implemented using the 'extends' keyword.

Types of Inheritance:

7. Single Inheritance: A class inherits from only one parent class. Example: `class Dog extends Animal {}`
8. Multilevel Inheritance: A class inherits from a class which itself inherits from another class. Example: A extends B, B extends C. Forms a chain.
9. Hierarchical Inheritance: Multiple classes inherit from a single parent class. Example: Dog, Cat, Horse all extend Animal.
10. Multiple Inheritance: A class inherits from more than one class. Java does NOT support multiple inheritance with classes to avoid diamond problem. Java supports it through interfaces.
11. Hybrid Inheritance: Combination of two or more types of inheritance. Supported through interfaces in Java.

Example of Single Inheritance:

```
class Animal { void eat() { System.out.println("Eating"); } }
class Dog extends Animal {
    void bark() { System.out.println("Barking"); }
}
Dog d = new Dog(); d.eat(); d.bark();
```

Q3. Write a Java program to print all Prime numbers from a class of integers.

```
class PrimeNumbers {
    int[] numbers;
    PrimeNumbers(int[] nums) {
        this.numbers = nums;
    }
    boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) return false;
        }
        return true;
    }
    void printPrimes() {
        System.out.print("Prime Numbers: ");
        for (int num : numbers) {
            if (isPrime(num))
                System.out.print(num + " ");
        }
    }
    public static void main(String[] args) {
        int[] arr = {2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 13};
        PrimeNumbers p = new PrimeNumbers(arr);
        p.printPrimes();
        // Output: Prime Numbers: 2 3 5 7 11 13
    }
}
```

Q4. Describe events and states. How is a state diagram prepared?

Event: An event is a specification of a significant occurrence that has a location in time and space. In UML, events trigger transitions between states. Types of events include: Signal events, Call events, Time events, and Change events.

State: A state represents a condition or situation during the life of an object during which it satisfies some condition, performs some activity, or waits for some event. Each state has a name, entry/exit actions, and internal transitions.

State Diagram Preparation:

12. Identify all possible states of the object (e.g., for an ATM: Idle, Card Inserted, PIN Entered, Transaction, Dispensing Cash).
13. Identify the initial state and final state.
14. Identify events that cause transitions between states.

15. Draw states as rounded rectangles.
16. Draw transitions as arrows labeled with events.
17. Mark the initial state with a filled circle and the final state with a bull's eye.

Example: For a traffic light system, states are Red, Yellow, Green and transitions occur on timer events.

Q5. What is meant by bytecode? Briefly explain how Java is platform independent.

Bytecode: When a Java source file (.java) is compiled using the Java compiler (javac), it is converted into an intermediate representation called bytecode (.class file). Bytecode is not machine-specific native code; it is a low-level set of instructions for the Java Virtual Machine (JVM).

Platform Independence in Java:

Java follows the principle of 'Write Once, Run Anywhere (WORA)'. The JVM is available for different operating systems (Windows, Linux, macOS). When a Java program is compiled, the bytecode is produced which is the same regardless of the platform. The JVM on the target machine interprets and executes this bytecode, translating it to native machine code.

Flow: Java Source Code → Java Compiler (javac) → Bytecode (.class) → JVM on any OS → Execution

Each OS has its own JVM implementation, but all JVMs understand the same bytecode format, ensuring platform independence.

Q6. Explain the significance of public, protected and private access specifiers in inheritance.

In Java inheritance, access specifiers control which members of the parent class are accessible in the child class:

public: Members declared public are accessible everywhere - in the same class, subclass, same package, and different package. Public members are fully inherited by subclasses.

protected: Members declared protected are accessible within the same package and in subclasses even if in a different package. Subclasses can access protected members of the parent class.

private: Members declared private are NOT accessible in subclasses. They belong only to the class in which they are defined. However, private members are inherited but cannot be accessed directly.

(default/package-private): Accessible within the same package only. Subclasses in the same package can access them, but subclasses in different packages cannot.

Q7. How does Polymorphism promote extensibility? Explain with example.

Polymorphism means 'many forms'. It allows the same interface to be used for different data types or objects. Polymorphism promotes extensibility because new classes can be added with the same interface without changing existing code.

Types: Compile-time (method overloading) and Runtime polymorphism (method overriding).

Example of Runtime Polymorphism:

```
class Shape {  
    void draw() { System.out.println("Drawing Shape"); }  
}
```

```
class Circle extends Shape {
    void draw() { System.out.println("Drawing Circle"); }
}
class Square extends Shape {
    void draw() { System.out.println("Drawing Square"); }
}
Shape s;
s = new Circle(); s.draw(); // Drawing Circle
s = new Square(); s.draw(); // Drawing Square
```

If a new shape (Triangle) is added, existing code does not change - just create a new subclass. This is extensibility through polymorphism.

Q8. Explain the process of defining and creating a package with suitable examples.

A package is a namespace that organizes related classes and interfaces. It avoids name conflicts and provides access control.

Step 1 - Define a Package:

```
// File: mypackage/Student.java
package mypackage;
public class Student {
    public void display() {
        System.out.println("Student class in mypackage");
    }
}
```

Step 2 - Compile with package structure:

```
javac -d . Student.java
```

Step 3 - Use the package in another class:

```
import mypackage.Student;
class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

The '-d .' flag tells the compiler to create the directory structure for the package.

Q9. Give an example where interface can be used to support multiple inheritance.

Java does not support multiple class inheritance but supports multiple inheritance through interfaces. A class can implement multiple interfaces.

```
interface Flyable {
    void fly();
}
interface Swimmable {
    void swim();
}
class Duck implements Flyable, Swimmable {
    public void fly() {
        System.out.println("Duck is flying");
    }
}
```



```
}
public void swim() {
    System.out.println("Duck is swimming");
}
}
class Main {
    public static void main(String[] args) {
        Duck d = new Duck();
        d.fly();    // Duck is flying
        d.swim();   // Duck is swimming
    }
}
```

Here Duck achieves multiple inheritance behavior by implementing both Flyable and Swimmable interfaces.

Q10. Differentiate between Checked and Unchecked Exceptions with examples.

Checked Exceptions: Exceptions that are checked at compile time. The programmer must either handle them using try-catch or declare them using the throws keyword. They are subclasses of Exception (but not RuntimeException). Examples: IOException, SQLException, FileNotFoundException.

```
// Checked Exception Example
try {
    FileReader fr = new FileReader("file.txt");
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
}
```

Unchecked Exceptions: Exceptions that are NOT checked at compile time. They occur at runtime. They are subclasses of RuntimeException. Examples: NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException.

```
// Unchecked Exception Example
int[] arr = {1, 2, 3};
System.out.println(arr[5]); // ArrayIndexOutOfBoundsException
```

Q11. Write a program to create four threads using Runnable interface.

```
class MyThread implements Runnable {
    String name;
    MyThread(String name) { this.name = name; }
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(name + " - Count: " + i);
            try { Thread.sleep(500); }
            catch (InterruptedException e) {}
        }
    }
}
public class FourThreads {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyThread("Thread-1"));
    }
}
```

```
Thread t2 = new Thread(new MyThread("Thread-2"));
Thread t3 = new Thread(new MyThread("Thread-3"));
Thread t4 = new Thread(new MyThread("Thread-4"));
t1.start(); t2.start(); t3.start(); t4.start();
}
}
```

Q12. Differentiate between ArrayList and Vector. Why is ArrayList faster than Vector?

ArrayList: Part of Java Collections Framework introduced in Java 2. Not synchronized (not thread-safe). Grows by 50% of its size when full. No overhead of synchronization. Faster in single-threaded environments. Methods are not synchronized.

Vector: Legacy class available since Java 1.0. Synchronized (thread-safe). Grows by 100% (doubles size) when full. Has overhead of synchronization. Slower due to synchronization overhead. Methods are synchronized.

Why ArrayList is faster: ArrayList's methods are not synchronized, meaning there is no locking mechanism. Vector acquires a lock on every read/write operation even when not needed in a single-threaded environment. This synchronization overhead makes Vector slower. For thread-safe operations, use `Collections.synchronizedList(new ArrayList<>())` or `CopyOnWriteArrayList`.

Q13. Distinguish between ByteStream Classes and Character Stream Classes.

Byte Stream Classes: Handle I/O of raw binary data (8-bit bytes). Suitable for all types of data including images, audio, and binary files. Base classes are `InputStream` and `OutputStream`. Examples: `FileInputStream`, `FileOutputStream`, `BufferedInputStream`, `DataInputStream`.

Character Stream Classes: Handle I/O of character data (16-bit Unicode characters). Suitable for text data only. Automatically handle character encoding and decoding. Base classes are `Reader` and `Writer`. Examples: `FileReader`, `FileWriter`, `BufferedReader`, `BufferedWriter`.

Key Difference: Byte streams are lower-level and handle raw bytes, while character streams handle text and support internationalization through Unicode. Character streams are preferred for text file processing.

Q14. Differentiate between multiprocessing and multithreading.

Multiprocessing: Execution of multiple processes simultaneously, each with its own memory space. Processes are independent and do not share memory. Communication between processes (IPC) is expensive. Each process has its own CPU overhead. Failure of one process does not affect others. Example: Running a browser and a media player simultaneously.

Multithreading: Multiple threads of the same process run concurrently. Threads share the same memory space (heap). Thread communication is easier and faster (shared memory). Less CPU overhead to create threads than processes. One thread failure can affect the entire process. Example: A web server handling multiple client requests using threads.

SECTION - C (Long Answer Questions)

(Each question carries 10 Marks)

Q1. Discuss Object Identity and Information Hiding under OOP.

Object Identity: Object identity is the property that distinguishes one object from another. Every object in an OOP system has a unique identity, even if two objects have identical attribute values. Identity is typically represented by a unique memory address or reference. In Java, the '==' operator compares references (identity), while the '.equals()' method compares values (equality). Object identity allows the same object to be referenced by multiple variables.

Information Hiding: Information hiding is the principle of concealing the internal implementation details of an object from the outside world. It is achieved through encapsulation using access modifiers (private, protected). Only necessary information is exposed through public interfaces.

Benefits of Information Hiding:

- Reduces complexity by hiding unnecessary details.
- Prevents unauthorized modification of data.
- Allows internal implementation to change without affecting users.
- Promotes modularity and maintainability.

Example: A BankAccount class hides the balance variable (private) and provides deposit() and withdraw() methods (public) to interact with it.

```
class BankAccount {  
    private double balance = 0;  
    public void deposit(double amt) { balance += amt; }  
    public void withdraw(double amt) {  
        if(amt <= balance) balance -= amt;  
    }  
    public double getBalance() { return balance; }  
}
```

Q2. Discuss the concept of passing arguments to methods using suitable examples.

In Java, arguments can be passed to methods in two ways:

1. **Pass by Value:** Java always passes arguments by value. For primitive types (int, float, etc.), a copy of the value is passed. Changes made to the parameter inside the method do not affect the original variable.

```
void change(int x) { x = 100; }  
int a = 10;  
change(a);  
// a is still 10 after the call
```

2. **Pass by Reference (Object Reference by Value):** For objects, the reference (address) is passed by value. Both the original and the parameter point to the same object. Changes to the object's state inside the method ARE reflected outside. However, reassigning the parameter does not affect the original reference.

```
void modifyArray(int[] arr) { arr[0] = 99; }  
int[] nums = {1, 2, 3};  
modifyArray(nums);  
// nums[0] is now 99
```

Variable-length arguments (varargs) allow passing variable number of arguments:

```
void printAll(String... args) {  
    for(String s : args) System.out.println(s);  
}  
printAll("Hello", "World", "Java");
```

Q3. What do you mean by Exceptions? Explain with suitable examples.

An Exception is an abnormal event that occurs during program execution and disrupts the normal flow of instructions. Java has a robust exception handling mechanism using try, catch, finally, throw, and throws.

Exception Hierarchy: Throwable -> Error and Exception. Exception -> Checked and Unchecked (RuntimeException).

Exception Handling Keywords:

- try: Block of code that might throw an exception.
- catch: Block to handle the exception.
- finally: Block that always executes, whether exception occurs or not.
- throw: Used to explicitly throw an exception.
- throws: Declares that a method can throw certain checked exceptions.

Example - Basic Exception Handling:

```
public class ExceptionDemo {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Exception: " + e.getMessage());
        } finally {
            System.out.println("Finally block executed");
        }
    }
}
```

Example - User-defined Exception:

```
class AgeException extends Exception {
    AgeException(String msg) { super(msg); }
}

class Test {
    static void checkAge(int age) throws AgeException {
        if(age < 18) throw new AgeException("Age must be >= 18");
        System.out.println("Valid Age");
    }
}
```

Q4. Explain in detail the concept, types and application of Polymorphism.

Polymorphism (from Greek: poly = many, morph = forms) is the ability of an object to take multiple forms. It allows the same interface to represent different underlying data types or implementations.

Types of Polymorphism:

1. Compile-time Polymorphism (Static Binding / Method Overloading):

Occurs when multiple methods have the same name but different parameters (different number, type, or order). Resolved at compile time.

```
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
```

}

2. Runtime Polymorphism (Dynamic Binding / Method Overriding):

Occurs when a subclass provides a specific implementation of a method defined in the parent class. Resolved at runtime using dynamic dispatch.

```
class Animal {  
    void sound() { System.out.println("Animal sound"); }  
}  
class Dog extends Animal {  
    void sound() { System.out.println("Dog barks"); }  
}  
Animal a = new Dog();  
a.sound(); // Output: Dog barks
```

Applications of Polymorphism:

- GUI frameworks use polymorphism to handle different event types uniformly.
- Database drivers implement a common interface (JDBC) with different backends.
- Allows writing generic code that works with future classes.
- Used in designing frameworks, plugins, and extensible architectures.

Q5. Describe Use cases, Use Case Diagrams and Activity Diagrams in Basic Structural Modeling.

Use Case: A use case describes a specific interaction between a user (actor) and the system. It represents a single unit of functionality from the user's perspective. Use cases help capture functional requirements. Each use case has a name, participating actors, preconditions, main flow, alternate flows, and postconditions.

Use Case Diagram: A UML diagram that visually represents the use cases, actors, and their interactions with the system. Components:

- Actor: External entity that interacts with the system (shown as stick figure).
- Use Case: Ellipses containing the name of functionality.
- System Boundary: Rectangle enclosing all use cases.
- Relationships: Association, Include, Extend, Generalization.

Example: For an ATM system, actors are Customer and Bank. Use cases are: Withdraw Cash, Check Balance, Change PIN, Deposit Cash.

Activity Diagram: A UML behavioral diagram that models the workflow or business process. It shows the sequence of activities, decisions, forks, and joins. Components:

- Initial Node: Filled circle - start of activity.
- Activity Node: Rounded rectangle - represents a single activity.
- Decision Node: Diamond - branching based on condition.
- Fork/Join: Horizontal bar - parallel activities.
- Final Node: Filled circle within a circle - end of activity.

Activity diagrams are useful for modeling business workflows, algorithms, and complex operations.

Q6. Briefly discuss Reusability, Extensibility, Robustness, and Abstraction.

- i) Reusability: The ability to use existing code (classes, methods, modules) in new programs without rewriting. Achieved through inheritance, composition, and packages in OOP. Benefits: reduces development time, reduces testing effort, improves reliability. Example: Java's standard library classes (String, ArrayList) are reused across millions of programs.
- ii) Extensibility: The ability to extend existing code to add new features without modifying existing code. Achieved through inheritance and polymorphism. The Open/Closed Principle states software should be open for extension but closed for modification. Example: Adding a new shape class (Triangle) without changing existing Shape hierarchy.
- iii) Robustness: The ability of software to function correctly under abnormal or unexpected conditions. Achieved through proper exception handling, input validation, defensive programming, and testing. Java's strong type checking, exception handling, and automatic garbage collection contribute to robustness.
- iv) Abstraction: The process of hiding complex implementation details and showing only the essential features of an object. Achieved through abstract classes and interfaces in Java. Abstraction reduces complexity, isolates the impact of changes, and allows the programmer to focus on 'what' rather than 'how'. Example: A car driver uses the steering wheel (abstract interface) without knowing the internal engine mechanics.

Q7. What is the difference between Function Overloading and Function Overriding?

Function Overloading (Compile-time Polymorphism): Defining multiple methods with the same name but different parameter lists within the same class. The correct method is selected at compile time based on the arguments. Return type alone cannot differentiate overloaded methods.

```
class Math {  
    int square(int x) { return x * x; }  
    double square(double x) { return x * x; }  
}
```

Function Overriding (Runtime Polymorphism): Redefining a method in a subclass that already exists in the parent class with the same name and parameters. The child class provides its own implementation. The method to be called is determined at runtime based on the actual object type.

```
class Vehicle { void move() { System.out.println("Vehicle moves"); } }  
class Car extends Vehicle {  
    void move() { System.out.println("Car drives"); }  
}
```

Key Differences: Overloading occurs within the same class; overriding occurs between parent and child classes. Overloading is resolved at compile time; overriding at runtime. Overriding requires same method signature; overloading requires different parameters.

Q8. What is Comparable and Comparator interface? Differentiate between them.

Comparable Interface (java.lang): Used to define the natural ordering of objects. A class implements Comparable<T> and overrides the compareTo(T obj) method. Used by Collections.sort() and Arrays.sort() by default.

```
class Student implements Comparable<Student> {  
    int marks;  
    public int compareTo(Student s) {  
        return this.marks - s.marks;  
    }  
}
```

```
}
```

Comparator Interface (java.util): Used to define custom/multiple orderings of objects. A separate class implements `Comparator<T>` and overrides `compare(T o1, T o2)`. Passed as argument to sort methods.

```
class NameComparator implements Comparator<Student> {  
    public int compare(Student s1, Student s2) {  
        return s1.name.compareTo(s2.name);  
    }  
}
```

Differences: Comparable modifies the original class; Comparator is external. Comparable provides one ordering; Comparator can provide multiple orderings. Comparable uses `compareTo()`; Comparator uses `compare()`.

Q9. What is Type Wrapper? What is the role of Autoboxing?

Type Wrapper: Wrapper classes provide an object representation for each of the eight primitive data types in Java. They 'wrap' the primitive values inside an object. Wrapper classes are in the `java.lang` package.

Primitive -> Wrapper class: `int` -> `Integer`, `double` -> `Double`, `char` -> `Character`, `boolean` -> `Boolean`, `byte` -> `Byte`, `short` -> `Short`, `long` -> `Long`, `float` -> `Float`.

Wrapper classes are used when: An object is required instead of a primitive (e.g., in Collections), converting between data types (`parseInt`, `toString`), and using utility methods (`max`, `min`, `parseInt`).

Autoboxing: Autoboxing is the automatic conversion of a primitive type to its corresponding wrapper class object. This was introduced in Java 5.

```
Integer i = 10; // Autoboxing: int -> Integer
```

Unboxing: The reverse process - automatic conversion of a wrapper object to its primitive type.

```
int x = i; // Unboxing: Integer -> int
```

Autoboxing and unboxing simplify code and eliminate the need for explicit conversion calls like `Integer.valueOf(10)` or `i.intValue()`.

Q10. What is Spring Boot and What are its Main Features?

Spring Boot is an open-source, Java-based framework built on top of the Spring Framework. It simplifies the process of creating stand-alone, production-grade Spring applications by providing auto-configuration, embedded servers, and opinionated defaults.

Main Features of Spring Boot:

18. Auto-Configuration: Automatically configures Spring application based on jar dependencies on the classpath.
19. Spring Boot Starters: Pre-defined dependency descriptors for common application types (web, data, security).
20. Embedded Server: Includes Tomcat, Jetty, or Undertow; no need to deploy WAR files.
21. Spring Boot CLI: Command-line interface to quickly create Spring Boot applications.
22. Actuator: Provides production-ready features like health checks, metrics, and monitoring endpoints.
23. Spring Initializr: Web-based tool to generate Spring Boot project scaffolding.
24. No XML Configuration: Relies on annotations and Java-based configuration.

25. Externalized Configuration: Supports application.properties / application.yml for configuration.

Q11. What are the Differences Between Spring and Spring Boot?

Spring Framework: A comprehensive Java framework for building enterprise applications. Requires extensive configuration (XML or Java-based). Developer must configure each component manually. No embedded server; requires external application server (Tomcat, JBoss). Complex setup for new projects. Provides features like IoC, AOP, MVC, Data Access.

Spring Boot: An extension of Spring that eliminates boilerplate configuration. Provides auto-configuration based on classpath dependencies. Comes with embedded servers (Tomcat). Enables rapid application development. Production-ready with Actuator. Reduces development time significantly.

In summary: Spring Boot is built on top of Spring and makes it easier to use by providing smart defaults, auto-configuration, and embedded infrastructure, whereas Spring requires manual configuration and setup.

Q12. How Can We Set Up a Spring Boot Application With Maven?

Step 1: Create a Maven project with the following pom.xml structure:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.0.0</version>
</parent>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
</dependencies>
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

Step 2: Create the main application class:

```
@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

Step 3: Run using mvn spring-boot:run or run the main class directly. The application starts on port 8080 by default.

Q13. What is Spring Initializr?

Spring Initializr (<https://start.spring.io>) is a web-based tool provided by the Spring team to quickly generate a Spring Boot project structure. It allows developers to: Select the project type (Maven/Gradle), choose the programming language (Java/Kotlin/Groovy), select the Spring Boot version, add dependencies (starters) with a search interface, and download a pre-configured project as a ZIP file. Spring Initializr saves time by generating the boilerplate project structure, pom.xml or build.gradle, and main application class automatically.

Q14. Explain Final, Finally and Finalize in Java.

final keyword: A non-access modifier that restricts modification. When applied to a variable, its value cannot be changed (constant). When applied to a method, it cannot be overridden by subclasses. When applied to a class, it cannot be inherited (subclassed). Example: `final int MAX = 100; final class String {} final void display() {}`

finally block: A block associated with try-catch that always executes regardless of whether an exception occurs or not. Used for cleanup code like closing database connections, file streams, etc.

```
try { /* code */ }
catch(Exception e) { /* handle */ }
finally { /* always executes */ }
```

finalize() method: A method defined in the Object class that is called by the Garbage Collector just before an object is garbage collected. Used to perform cleanup operations on the object. It is deprecated since Java 9 and should not be relied upon.

```
protected void finalize() { /* cleanup */ }
```

Q15. What Spring Boot Starters Are Available?

Spring Boot Starters are dependency descriptors in Maven/Gradle that simplify adding pre-configured groups of dependencies. Major starters include:

26. spring-boot-starter-web: For building web and RESTful applications using Spring MVC with Tomcat.
27. spring-boot-starter-data-jpa: For JPA-based data access with Hibernate.
28. spring-boot-starter-security: For Spring Security (authentication and authorization).
29. spring-boot-starter-test: For testing with JUnit, Mockito, and Spring Test.
30. spring-boot-starter-thymeleaf: For Thymeleaf template engine integration.
31. spring-boot-starter-actuator: For production monitoring and management.
32. spring-boot-starter-mail: For email sending via JavaMail.
33. spring-boot-starter-cache: For enabling caching support.
34. spring-boot-starter-validation: For bean validation with Hibernate Validator.
35. spring-boot-starter-websocket: For WebSocket-based messaging.

Q16. What is an Interface? What are the similarities between interfaces and classes?

Interface: An interface in Java is a reference type, similar to a class, that can contain only abstract methods (prior to Java 8), static methods, default methods (Java 8+), and constants. Interfaces define a contract that implementing classes must fulfill. Declared using the 'interface' keyword. A class implements an interface using 'implements' keyword. Interfaces are used to achieve abstraction and multiple inheritance.

```
interface Drawable {  
    void draw(); // abstract method  
    default void info() { System.out.println("Drawable interface"); }  
}
```

Similarities between Interfaces and Classes:

- Both can have methods and variables.
- Both are compiled into .class (bytecode) files.
- Both can be used as data types (polymorphism).
- Both can extend other types (interface extends interface; class extends class).
- Both can contain constants.
- Both can have generic type parameters.

Q17. Write the significance of Java Virtual Machine (JVM).

The Java Virtual Machine (JVM) is an abstract computing machine that enables a computer to run Java programs. It is the core of Java's platform independence.

Significance of JVM:

36. Platform Independence: JVM interprets Java bytecode and converts it to native machine code for the underlying OS. The same bytecode runs on any machine with JVM installed.
37. Memory Management: JVM manages memory allocation for objects and performs automatic garbage collection, freeing unused memory.
38. Security: JVM acts as a security sandbox. It verifies bytecode before execution to prevent malicious code.
39. Performance Optimization: JVM includes a Just-In-Time (JIT) compiler that compiles frequently used bytecode to native code at runtime for better performance.
40. Exception Handling: JVM provides built-in support for exception handling.
41. Class Loading: JVM's ClassLoader dynamically loads classes at runtime when needed.

JVM Architecture: ClassLoader -> Bytecode Verifier -> Interpreter/JIT Compiler -> Runtime Data Areas (Heap, Stack, Method Area, etc.) -> Execution Engine -> Native Method Interface.

--- END OF SOLUTION MANUAL ---
